

Axios – Full Production Roadmap

Real-time spoilage intelligence for perishable goods (paneer, SGP41-based)

This is a no-shortcuts build plan for Phases 10–20: ingestion through dashboard, with validation baked in rather than bolted on at the end. It assumes AWS IoT Core is already publishing telemetry (confirmed live) and treats the sensor's own `voc_index` / `nox_index` as unreliable (both are pinned at 0 in your current stream, consistent with the SGP41 gas-index algorithm not having completed its conditioning window). Everything here is built on `voc_raw` / `nox_raw` plus your own per-session baseline.

1. Product framing (read this before writing any code)

Axios answers three questions for one shipment, continuously:

1. **How degraded is it right now?** (biochemical + thermal risk, decomposed and shown separately)
2. **When did degradation start?** (onset detection, timestamped, with confidence)
3. **How much time is left?** (remaining useful life, as a range, not a false-precision number)

Everything downstream of that must be defensible: every risk number should trace back to raw telemetry through an inspectable chain (raw → filtered → derivative → signal → score → fusion), and every state transition should be evidence-backed and replayable from stored data. That traceability *is* the product — it's what makes this different from a generic IoT dashboard, and it directly serves the dispute/evidence use case in the Product X material.

Non-negotiables carried through the whole roadmap:

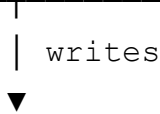
- **Never discard raw data.** Every transformation stores its output alongside — never over — the input.
 - **No stage silently fails.** Missing/invalid data produces an explicit `INSUFFICIENT_DATA` state, never a fabricated number.
 - **No score is presented as more certain than it is.** Risk scores stay risk scores until Phase 20 calibration is actually done against ground truth.
-

2. System architecture

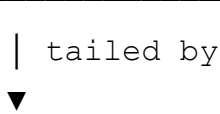
```
|
|   AWS IoT Core (MQTT)
|   freshtrace/device/*/...
|
```



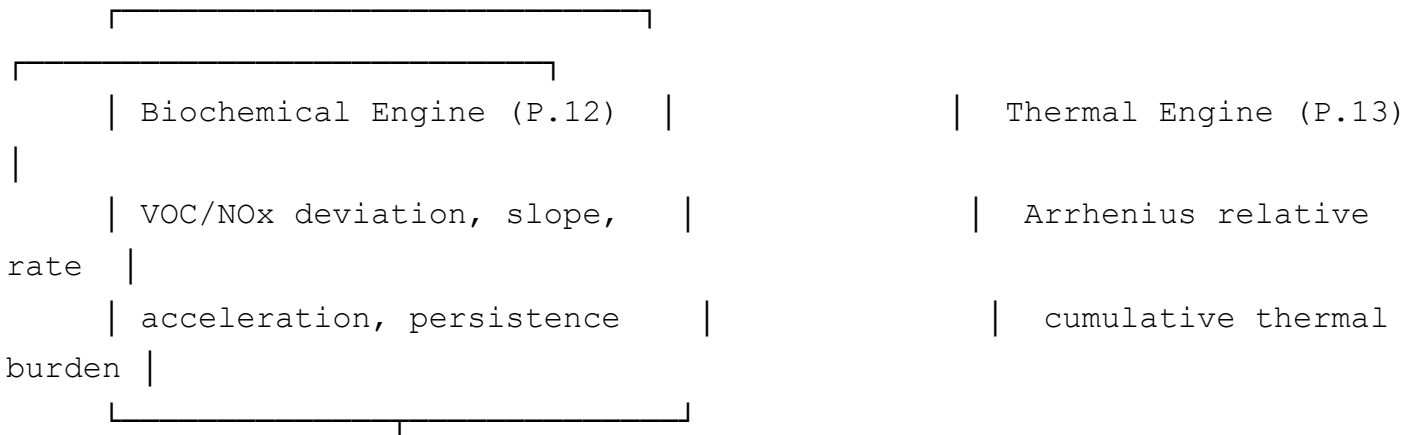
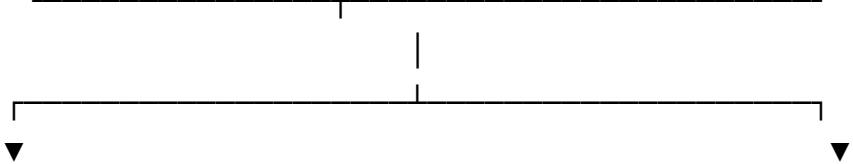
```
|
|   Ingestion Service (Phase 10)
|   - MQTT subscriber (persistent)
|   - schema validation
|   - dedup / ordering
|   - local disk buffer (outage-safe)
|
```

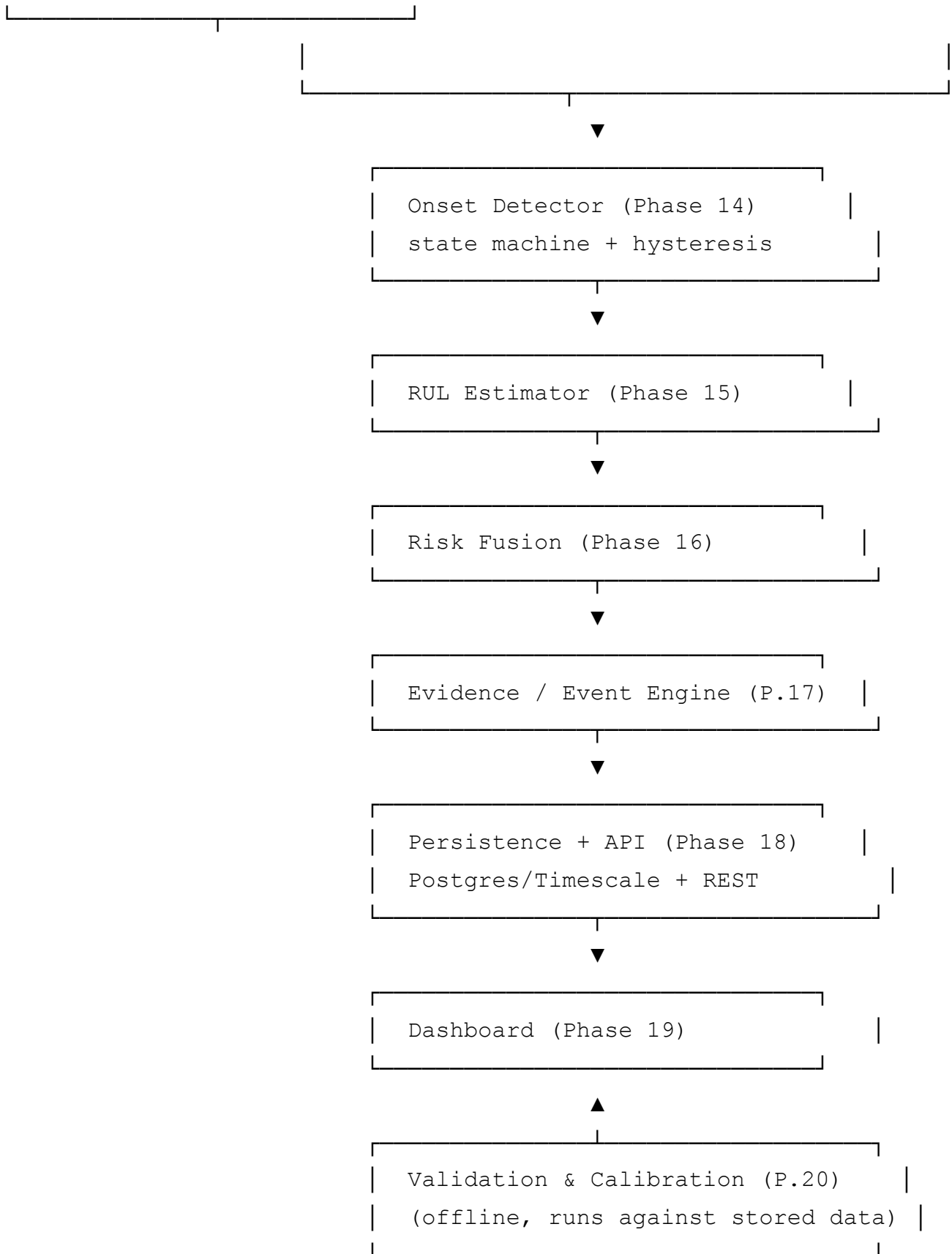


```
|
|   Raw Telemetry Store
|   (append-only, per device/session)
|   data/raw/telemetry/*.jsonl
|
```



```
|
|   Processing Service (Phase 11)
|   - session manager
|   - baseline engine
|   - filters (median, EMA, low-pass)
|   - derivatives (velocity, acceleration)
|
```





Why a store-then-process split (not a pure streaming pipeline): every stage needs to be replayable against historical sessions for debugging, demo, and Phase 20 calibration. If processing only ever happens in-flight on a live MQTT message, you lose the ability to re-run Phase 12's scoring logic against last week's data after you improve it. Raw telemetry is the source of truth; everything else is a recomputable view over it.

Process boundaries: Ingestion, Processing, and the two scoring engines should be separate services (or at minimum separate modules with no shared mutable state) that communicate by reading/writing the telemetry and session stores — not by passing objects in memory. This means you can restart the biochemical engine after a bug fix and have it recompute from stored raw data without re-acquiring sensor connectivity.

3. Data flow, end to end

1. Device publishes to `freshtrace/device/{device_id}/telemetry` roughly every few seconds (your capture shows ~1 MQTT publish per 3 `[SENSORS]` log lines — confirm the actual interval against wall-clock timestamps before setting any time-based thresholds later).
 2. Ingestion service receives, validates against schema, timestamps with **receipt time** (not device time, unless device clock sync is verified), and appends to the raw store as one JSON line per reading.
 3. A session is either continued (device already has an open session) or started (first reading for a new shipment). Session boundaries are explicit — a human or an API call marks "shipment started," not an inferred gap in data.
 4. Processing service tails new raw readings, updates the running baseline, filters, computes derivatives, and writes a `ProcessedReading` record.
 5. Biochemical and thermal engines run independently off `ProcessedReading` history (not off each other) and produce a `BiochemicalScore` and `ThermalScore` per reading (or per small batch, see §5.4 on cadence).
 6. Onset detector consumes the biochemical + thermal score stream and maintains one state machine instance per session.
 7. RUL estimator runs whenever the onset detector is past `WATCH`, using the degradation trajectory so far.
 8. Fusion combines thermal + biochemical into one risk number and a dominant-risk label.
 9. Evidence engine watches all of the above for state transitions and threshold crossings, and writes immutable event records.
 10. Everything above is persisted; the API reads from persisted state (never recomputes live on request) so response times stay flat and demo behavior is reproducible.
 11. Dashboard polls or subscribes to the API and renders current state + history.
-

4. Data schemas

4.1 Raw telemetry (as received, Phase 10 output)

```
{
  "device_id": "FreshTrace-Node-01",
  "session_id": "sess_2026-09-19_paneer_01",
  "received_at": "2026-09-19T14:02:03.114Z",
  "device_reported_at": null,
  "food_type": "paneer",
  "temperature_c": 30.20,
  "humidity_pct": 66.05,
  "pressure_hpa": 947.09,
  "voc_raw": 32158,
  "nox_raw": 21861,
  "voc_index": 0,
  "nox_index": 0,
  "wifi_rssi": -35,
  "sequence_number": 1042
}
```

`sequence_number` should be assigned by the ingestion service (monotonic per device) — this is what lets you detect drops and out-of-order delivery without trusting device-side timestamps.

4.2 Processed reading (Phase 11 output)

```
{
  "session_id": "sess_2026-09-19_paneer_01",
  "timestamp": "2026-09-19T14:02:03.114Z",
  "raw": { "voc": 32158, "nox": 21861, "temperature_c": 30.20, "humidity_":
  "filtered": { "voc": 32161.4, "nox": 21930.2, "temperature_c": 30.21 },
  "baseline": { "voc": 32150.0, "nox": 21980.0, "temperature_c": 30.15 },
  "voc_velocity": 0.8,
  "voc_acceleration": -0.1,
  "nox_velocity": -4.6,
  "nox_acceleration": -0.3,
  "voc_relative_change": 0.00034,
  "nox_relative_change": -0.00227,
  "reading_quality": "ok"
}
```

`reading_quality` is `ok` | `interpolated` | `rejected_outlier` | `insufficient_history` — every downstream engine checks this field rather than silently trusting filtered values.

4.3 Session record

```
{
  "session_id": "sess_2026-09-19_paneer_01",
  "device_id": "FreshTrace-Node-01",
  "food_type": "paneer",
  "started_at": "2026-09-19T14:00:00Z",
  "baseline_established_at": "2026-09-19T14:05:00Z",
  "status": "active",
  "baseline_window_readings": 60
}
```

4.4 State object (Phase 18 primary API object – matches your original spec, extended)

```
{
  "device_id": "FreshTrace-Node-01",
  "session_id": "sess_2026-09-19_paneer_01",
  "food_type": "paneer",
  "state": "DEGRADING",
  "spoilage_risk": 0.74,
  "spoilage_onset": { "detected": true, "timestamp": "2026-09-19T18:42:13" },
  "remaining_useful_life": { "hours": 38.4, "lower": 27.1, "upper": 55.8,
  "degradation_velocity": 0.038,
  "risk": { "thermal": 0.42, "biochemical": 0.81, "dominant": "biochemical" },
  "data_quality": { "last_reading_age_seconds": 4, "readings_in_session": 1,
  "last_updated": "2026-09-19T19:10:02Z"
}
```

`data_quality` matters for the UX (§9) – the dashboard should visibly degrade its confidence display when data is stale or sparse, rather than showing a two-hour-old risk number as if it were live.

5. Phase 10 – Ingestion (build on top of the existing AWS IoT connection)

You already have the MQTT connection; the work here is making it production-safe, not establishing it.

5.1 Subscriber

- Persistent MQTT client with automatic reconnect and exponential backoff.
- Subscribe with QoS 1 at minimum (at-least-once delivery) so a broker hiccup doesn't silently drop a reading.
- One process per device is fine at this scale; don't over-engineer horizontal scaling for a single-node demo, but don't hardcode the `device_id` either — read it from the topic.

5.2 Validation (reject or flag, never guess)

Explicit checks before a reading is accepted:

- All required fields present and correct type.
- `temperature_c` within a physically plausible range for the transport/storage context (e.g. -10°C to 60°C — wide enough to not choke on a legitimate excursion, narrow enough to catch sensor faults).
- `humidity_pct` in $[0, 100]$.
- `voc_raw`, `nox_raw` non-negative integers within the sensor's documented output range.
- `pressure_hpa` within a plausible range (roughly 850–1100 for a ground-level device).

A reading that fails validation is stored in a separate `rejected/` stream with the reason, not discarded — you want this for debugging sensor faults later, and it becomes useful evidence if a dispute ever hinges on "was the sensor even working."

5.3 Deduplication & ordering

- Use `sequence_number` (assigned at ingestion) to detect gaps and out-of-order arrival.
- On out-of-order arrival within a small window (a few seconds), re-sort before writing.
- On a detected gap, write a `GAP_DETECTED` marker into the raw stream rather than silently continuing — Phase 11's baseline and derivative math need to know a gap happened so they don't compute a false velocity across a 10-minute hole.

5.4 Local buffering

- If the raw store (disk/DB) is temporarily unwritable, buffer to a local in-memory queue with a disk-backed overflow (even a simple append-only file works) and flush on recovery.
- This is specifically about *your backend's* durability, separate from whatever buffering the device firmware already does for network loss to AWS IoT — don't conflate the two.

5.5 Output

- Append-only JSONL per session:
`data/raw/telemetry/{device_id}/{session_id}.jsonl`.
 - Also write to a queryable store (Postgres/Timescale table `raw_telemetry`) for the processing service to tail – flat files are your audit trail, the DB is what the rest of the system reads from.
-

6. Phase 11 – Session baseline & signal processing

6.1 Session lifecycle

SESSION START (explicit, via API call or device pairing event)

|



collect baseline readings (fixed window, e.g. first 60 readings or first 10 minutes, whichever is longer)

|



baseline established (mean + std dev per channel, stored on the session record)

|



monitor degradation (every subsequent reading compared against this baseline)

A baseline window that's too short will anchor on transient startup noise (sensor warm-up drift is real and documented for SGP41-class sensors – expect the first several minutes of readings to be less stable). Don't start scoring degradation until the baseline window has closed; readings during that window get `reading_quality: "insufficient_history"` and are excluded from risk scoring, though still stored.

Baseline is per-session, per-device – never a global constant across shipments, since ambient VOC/NOx floor varies with the specific transport environment.

6.2 Noise filtering

Maintain, per channel (VOC, NOx, temperature):

- **Moving median** (window of ~5 readings) – cheap, kills single-sample spikes without smearing real step changes.
- **Exponential moving average** on top of the median-filtered value, for the smoother trend line the biochemical/thermal engines actually consume:

$$\text{EMA}_t = \alpha \cdot x_t + (1 - \alpha) \cdot \text{EMA}_{t-1}$$

Pick α empirically against your real noise floor — from your captured data, raw VOC jitters by roughly 50–100 counts against a 32,000 baseline (0.2–0.3%), and NOx by a comparable relative amount. Start with $\alpha \approx 0.2-0.3$ (smooths over ~5–8 readings) and tune against how it looks on your captured trace before locking it in.

- **Outlier rejection:** reject a reading if it deviates from the current EMA by more than N standard deviations of the recent residual (computed from the last ~50 readings) *and* the jump doesn't persist on the next reading. A real step change (e.g. thermal excursion) should be re-confirmed by the following reading; a single-sample spike won't be.
- **Rate-of-change limit:** flag (don't discard) a reading whose implied instantaneous rate of change exceeds a physically implausible bound (e.g. temperature can't jump 10°C in one reading interval under normal transport).

Store `raw_value` and `filtered_value` side by side, always. Never overwrite raw.

6.3 Derivatives

For VOC and NOx (using the filtered value):

$$\text{velocity}_t = (x_t - x_{t-1}) / \Delta t$$

$$\text{acceleration}_t = (v_t - v_{t-1}) / \Delta t$$

Compute these on the filtered series, not raw — otherwise sensor jitter dominates the derivative (derivatives amplify noise, which is exactly why the filtering step has to happen first).

This distinction — *high VOC* vs. *VOC rapidly increasing* — is what onset detection in Phase 14 actually keys off, so get this right before moving on; it's the single most important computed signal in the whole pipeline.

6.4 Relative deviation

$$D(t) = (x(t) - x_0) / (|x_0| + \varepsilon)$$

where x_0 is the session baseline mean for that channel. This normalizes across devices/sessions that might have different ambient floors, and is what feeds `voc_relative_change` / `nox_relative_change` in the schema above. Use a small ε (e.g. 1.0) purely to avoid divide-by-zero on a pathological baseline of 0 — it shouldn't meaningfully affect the math at your real value ranges.

7. Phase 12 — Biochemical Degradation Engine

Explicitly not ML — a deterministic, inspectable function of engineered signals. This is a feature, not a limitation: every number here is traceable back to a specific sensor reading.

7.1 Per-channel signal computation (VOC and NOx, computed identically)

- **Deviation** D : relative deviation from baseline (§6.4).
- **Slope** S : normalized velocity — velocity divided by baseline magnitude, so it's comparable across channels with different absolute scales.
- **Acceleration** A : normalized acceleration, same treatment.
- **Volatility**: rolling standard deviation of the filtered signal over a recent window (e.g. last 30 readings) — distinguishes a steady trend from an erratic one.
- **Persistence** P : fraction of the last N readings where deviation has been consistently in the "degrading" direction (not just currently elevated). This is what prevents a single noisy excursion from reading as sustained degradation.

7.2 Multi-signal biochemical score

$$B(t) = w1 \cdot D_{voc} + w2 \cdot D_{nox} + w3 \cdot S_{voc} + w4 \cdot S_{nox} + w5 \cdot A_{voc} + w6 \cdot A_{nox} + w7 \cdot P$$

Start with equal or near-equal weights, then adjust based on which channel your captured data shows moving first/most reliably (your sample data shows NOx declining steadily while VOC stays roughly flat — if that pattern holds over a longer capture, NOx deviation/slope may deserve more weight for paneer specifically; don't assume this generalizes to other food types without separate calibration).

Clip $B(t)$ to $[0, 1]$ via a bounded transform (e.g. $1 - e^{-k \cdot B_{raw}}$) rather than a hard clamp, so the score saturates smoothly instead of pinning at an arbitrary boundary.

Important scope note: a declining NOx signal and a rising NOx signal likely mean different things biologically (NOx trends can go either direction depending on the specific spoilage chemistry of the food type and what the sensor is actually responding to). Don't assume "any deviation = degradation" — this is exactly what Phase 20 validation is for. For the initial build, treat *magnitude* of sustained deviation from baseline as the risk driver regardless of direction, and revisit the sign convention once you have labeled spoilage data.

7.3 Output

```
{ "biochemical_risk": 0.64, "voc_signal": 0.71, "nox_signal": 0.52, "degr
```

Store the per-channel signals alongside the fused score — the dashboard's risk decomposition view (§9) and the evidence engine both need them individually, not just the combined number.

8. Phase 13 – Thermal-Kinetic Engine

8.1 Temperature history

Maintain, per session: current, running average, running maximum, cumulative time above a configured excursion threshold, and current excursion duration (reset to 0 when temperature drops back below threshold, but log the completed excursion as an event first).

8.2 Arrhenius relative rate

$$k(T) = A \cdot e^{(-E_a / (R \cdot T))}$$

$$k(T) / k(T_{\text{ref}}) = e^{-(E_a/R) \cdot (1/T - 1/T_{\text{ref}})}$$

Temperature in Kelvin. E_a (activation energy) and T_{ref} (reference temperature, typically your target storage temp) must be **configurable per food type**, stored in a config file/table, not hardcoded – paneer's real degradation kinetics differ from produce or meat, and you shouldn't need a code change to add a second food type. For an initial paneer config, use published dairy-spoilage activation energy ranges as a starting point and flag clearly in the config that these are literature-sourced defaults pending your own calibration (Phase 20).

8.3 Cumulative thermal exposure

$$E_{\text{thermal}} = \int k_{\text{relative}}(T(t)) dt$$

Implement as a running numerical integral (trapezoidal rule between consecutive readings is sufficient given your sampling rate) rather than a simple sum, so uneven reading intervals (gaps, decimation) don't distort the integral.

8.4 Output

```
{ "thermal_risk": 0.72, "thermal_exposure": 14.3, "temperature_excursion"
```

thermal_risk is E_{thermal} normalized against a configured critical exposure threshold (again per food type), saturating smoothly like the biochemical score.

9. Phase 14 – Spoilage Onset Detector

A state machine, not a single threshold – this is the difference between "the sensor twitched" and "the product is actually degrading."

STABLE

| persistent biochemical deviation (D above threshold for $\geq$ persistence window)



WATCH

| sustained signal AND positive acceleration



ONSET CANDIDATE

| hysteresis: signal stays above (threshold - margin) for the full persistence window,

| AND signal-to-noise ratio $\geq$ minimum



SPOILAGE ONSET (timestamped, confidence assigned)



DEGRADING

9.1 Parameters (all configurable, not hardcoded)

- `deviation_threshold`: minimum $D(t)$ to leave STABLE.
- `persistence_window`: number of consecutive qualifying readings required – **size this in readings actually received, calibrated against your confirmed real publish interval**, not an assumed cadence. If your device publishes roughly every few seconds, a 2-minute persistence requirement is a specific, known number of readings; get the real interval confirmed before hardcoding it.
- `hysteresis_margin`: how far the signal must fall before a state can regress (prevents flapping right at the threshold boundary).
- `min_snr`: minimum ratio of signal magnitude to recent volatility (§7.1) before a transition is allowed – this is what stops a single loud-but-noisy reading from triggering onset.

9.2 Confidence

`onset_confidence` is a function of: how far past threshold the signal is, how long it persisted, and the SNR at the time of transition – not a separate model, just a documented weighted combination of numbers you already computed. Keep it simple and explainable; this is exactly the kind of score that should stay an "engineered confidence" per the original spec until Phase 20 calibrates it against real observed onset times.

9.3 Output

```
{ "onset_detected": true, "onset_timestamp": "2026-09-19T18:42:13Z", "ons
```

10. Phase 15 – Remaining Useful Life Engine

Don't ship a single-derivative estimate as the only method – it's fragile to noise right at the moment you need it most. Build it in two layers:

10.1 Layer 1 – trajectory-based estimate (primary)

Fit a simple regression (linear, or low-order polynomial if the trend is clearly curving) over the **recent degradation trajectory** (e.g. last 30–60 minutes of $B(t)$ and fused risk), not just the instantaneous derivative at the last point:

$$T_{\text{remaining}} \approx (D_{\text{critical}} - D(t)) / (dD/dt)_{\text{smoothed}}$$

where $(dD/dt)_{\text{smoothed}}$ comes from the regression slope, not the raw last-two-points derivative.

10.2 Layer 2 – bounds from trajectory uncertainty

Compute the regression's confidence interval and propagate it into lower_bound_hours / upper_bound_hours . If you don't want to implement full regression confidence intervals in the time available, a defensible fallback is: run the same extrapolation using the fastest and slowest sustained slope observed in the recent window, and report those as the bounds. Either way, **never output a bare point estimate with no range** – the whole point of showing RUL as $\text{hours} / \text{lower} / \text{upper}$ is honesty about uncertainty, and that's a UX commitment as much as a math one (see §9's dashboard framing).

10.3 Incorporate thermal exposure

$T_{\text{remaining}}$ should shorten as thermal_risk rises, not just track the biochemical trajectory in isolation – a shipment sitting at an elevated temperature will degrade faster than its current biochemical slope alone predicts. A simple, defensible way to do this without a full joint model: scale the biochemical slope by the current Arrhenius relative rate $k(T) / k(T_{\text{ref}})$ before extrapolating.

10.4 Output

```
{ "remaining_useful_life_hours": 38.4, "lower_bound_hours": 27.1, "upper_
```

Label this on the dashboard as **"Estimated Remaining Useful Life"** – never "shelf life" – exactly as the original spec insists. This isn't just a legal hedge; it's accurate to what the number actually is.

11. Phase 16 – Thermal vs. Biochemical Risk Fusion

```
overall_risk = w_thermal * thermal_risk + w_biochemical * biochemical_risk
dominant_risk = argmax(thermal_risk, biochemical_risk)
```

Start with equal weights; revisit once Phase 20 gives you evidence about which channel is actually more predictive for paneer specifically.

11.1 State labels

- `THERMAL_DOMINANT` – thermal_risk substantially exceeds biochemical_risk.
- `BIOCHEMICAL_DOMINANT` – reverse.
- `DUAL_RISK` – both elevated and close together.
- `LOW_RISK` – both below their respective thresholds.
- `INSUFFICIENT_DATA` – baseline not yet established, or a data gap is currently active.

This last state matters more than it looks – it's the mechanism that stops the dashboard from ever showing a confident-looking risk number computed from three stale or missing readings.

12. Phase 17 – Evidence & Event Engine

Every state transition and threshold crossing becomes an immutable, timestamped record – this is what makes Axios auditable rather than just a live gauge.

Event types: `SESSION_STARTED`, `BASELINE_ESTABLISHED`,
`TEMPERATURE_EXCURSION_START/END`, `BIOCHEMICAL_ACCELERATION`,
`SPOILAGE_ONSET_CANDIDATE`, `SPOILAGE_ONSET_CONFIRMED`, `THERMAL_RISK_HIGH`,
`BIOCHEMICAL_RISK_HIGH`, `RUL_THRESHOLD_CROSSED`, `DATA_GAP_DETECTED`.

```
{
  "event": "BIOCHEMICAL_ACCELERATION",
  "session_id": "sess_2026-09-19_paneer_01",
  "timestamp": "2026-09-19T18:31:00Z",
```

```
"severity": 0.82,  
"evidence": {  
  "voc_slope": 0.021, "nox_slope": -0.045,  
  "persistence_seconds": 240, "temperature_c": 31.4  
}  
}
```

Events are append-only – never edited or deleted once written, even if a later state correction happens (a correction is itself a new event, referencing the one it supersedes). This is the property that makes the evidence timeline usable in an actual dispute.

13. Phase 18 – Persistence & API

13.1 Storage

- **Timescale/Postgres** for `raw_telemetry`, `processed_readings`, `sessions`, `state_snapshots`, `events` – time-series-friendly, queryable, and you'll want SQL for the Phase 20 validation analysis anyway.
- Keep the flat-file JSONL raw log too, as a redundant audit trail independent of the database.

13.2 Endpoints

```
GET /devices  
GET /devices/{device_id}/status  
GET /devices/{device_id}/telemetry?from=&to=  
GET /devices/{device_id}/risk  
GET /devices/{device_id}/events  
GET /devices/{device_id}/timeline  
GET /devices/{device_id}/report          (generates a shareable summary –  
see §14.5)  
POST /sessions                          (explicit session start)  
POST /sessions/{session_id}/close       (explicit session end)
```

The API reads persisted state – it does not recompute risk scores on request. This keeps latency flat and, critically, means the dashboard and any exported report always agree with what was actually stored as evidence, rather than a number that could shift between two requests a second apart.

14. Phase 19 – Dashboard & UX

14.1 Design principle

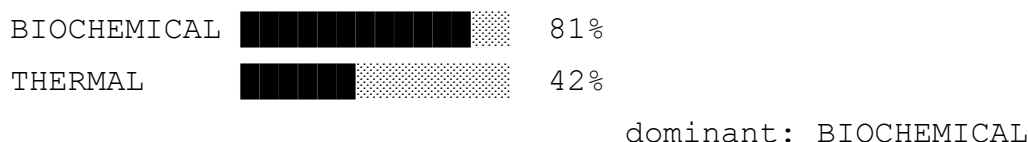
The dashboard's job is to make an operator trust a number they didn't compute themselves. That means: show the decomposition, not just the headline; make data staleness visible, not hidden; and never let a single number stand without the trend that produced it.

14.2 Primary view – Shipment Condition

FreshTrace-Node-01 · paneer
SHIPMENT CONDITION: HIGH RISK
Estimated Remaining Useful Life
38.4 h (range: 27.1 – 55.8 h)
Last reading: 4s ago ●live

The state label (**LOW_RISK** / **DUAL_RISK** / **DEGRADING** etc.) drives the color, not a separately-invented color scale – one source of truth for "how bad is this."

14.3 Risk decomposition (always visible, not behind a click)



This is non-negotiable given the product's whole differentiation is explaining *why*, not just *how much*.

14.4 Timeline

- Dual-axis chart: temperature history (line) and VOC/NOx trajectories (lines), with the onset timestamp marked as a vertical annotation.
- Degradation velocity shown as a small trend indicator: **stable** → **accelerating** → **rapid**, derived directly from the sign and magnitude of **degradation_velocity**, not a separate judgment call.
- Evidence timeline underneath as a scrollable list, each entry linked to the exact chart position:

14:02 baseline established
17:18 thermal excursion
18:31 VOC acceleration detected

18:42 biochemical onset detected

19:05 RUL threshold crossed

14.5 Data quality affordance

When `data_quality.last_reading_age_seconds` exceeds $\sim 2\times$ the expected publish interval, the "●live" indicator switches to a visibly stale state ("●last seen 4m ago") and the risk numbers get a subdued visual treatment (not hidden — still evidence, just flagged as not current). This one UI decision does more to earn operator trust than any amount of polish on the "good" state.

14.6 Report export

A shareable, static snapshot (PDF or shareable link) generated from `/devices/{id}/report` — the full risk breakdown, timeline, and evidence log for one session, frozen at export time. This is the artifact that actually gets used in a dispute, so it should be generated from persisted state, never recomputed at view time.

14.7 Multi-device view

A simple list/grid of all active devices with just device name, food type, state label, and RUL, sorted by risk descending — the entry point before drilling into the single-shipment view above.

15. Phase 20 — Validation & Calibration

Do not skip this even in a compressed timeline — it's what upgrades every "risk score" in this document into something you can eventually call a probability, and it's the only way to know if your weights (§7.2, §11) are anywhere close to right.

15.1 Protocol per food type

fresh sample → controlled storage → continuous SGP41 + temperature + humidity logging

→ periodic physical/quality assessment (visual, smell, or lab-verified spoilage marker)

→ recorded actual spoilage endpoint

15.2 Metrics

Onset detection: predicted onset timestamp vs. observed onset — track timing error (signed, so you know if the system runs early or late), false-positive rate (onset flagged with no real spoilage), false-negative rate (real spoilage with no flag).

RUL: predicted vs. observed remaining life — mean absolute error, whether the observed value actually falls inside the reported `[lower, upper]` interval (interval coverage — if your 90%-ish interval only contains the true value 40% of the time, the bounds method in §10.2 needs revisiting), and whether errors skew early or late.

Risk score → probability: only after enough labeled sessions exist, fit a calibration curve (even a simple logistic fit of `risk score → observed-spoiled-within-RUL-window`) and only then start describing `spoilage_risk` as a probability rather than a score. Keep both the raw engineered score and the calibrated probability in the schema once this exists — don't discard the raw score, since it's still useful for debugging.

15.3 Where this fits the timeline

Realistically, Phase 20 doesn't run as a discrete phase before launch — it runs continuously against every real session you log from day one. Build the event/evidence store (Phase 17) and persistence layer (Phase 18) with this in mind from the start: they need to retain enough raw + processed history per session that a calibration pass months later can be run without having planned for it in advance.

16. Project structure

```
Axios/
├─ data/
│   ├── raw/telemetry/{device_id}/{session_id}.jsonl
│   ├── processed/{telemetry,sessions}/
│   └─ events/
├─ src/
│   ├── ingestion/mqtt.py
│   ├── processing/{filters,baseline,derivatives,trajectory}.py
│   ├── biochemical/{voc,nox,degradation}.py
│   ├── thermal/{arrhenius,exposure}.py
│   ├── detection/onset.py
│   ├── rul/estimator.py
│   ├── fusion/risk.py
│   ├── evidence/events.py
│   └─ api/routes.py
└─ config/
```

```
|   └─ food_types/{paneer,...}.yaml      # Ea, T_ref, thresholds, weights
- per food type
└─ artifacts/configs/
└─ scripts/phase{10..18}/
└─ dashboard/
└─ reports/
```

The `config/food_types/` directory is the concrete implementation of "configurable, not hardcoded" from §8.2 and elsewhere — every constant this document flags as food-type-specific lives there, not inline in the engine code.

On Phases 1-9: keep them, but mark clearly as an experimental/research branch (Hohenheim data pipeline, ML experiments/evaluation/registry) — not used by production inference. The production path starts at Phase 10 on real SGP41 telemetry, for the reasons already established: a model trained on unrelated MQ-sensor channels doesn't transfer to SGP41 signal characteristics, so this roadmap deliberately keeps the production system as engineered signal processing rather than a repurposed ML model.

17. Build sequencing (no time-box — ordered by dependency, not by day)

1. **Ingestion hardening** (§5) — validation, sequencing, gap detection, local buffering. Everything downstream depends on trustworthy raw data.
2. **Session + baseline** (§6.1) — get this right before any filtering, since a bad baseline window silently corrupts every score computed against it.
3. **Filtering + derivatives** (§6.2-6.4) — tune against your actual captured noise floor, not assumed numbers.
4. **Biochemical + thermal engines in parallel** (§7-8) — independent enough to build simultaneously.
5. **Onset detector** (§9) — depends on both engines being stable first; tuning persistence windows before the upstream signals are trustworthy just wastes time re-tuning later.
6. **RUL + fusion** (§10-11).
7. **Evidence engine** (§12) — can be built incrementally alongside 4-6, since it's mostly listening for events already being computed.
8. **Persistence + API** (§13).
9. **Dashboard** (§14) — build against real persisted data from step 8 onward, not mocked data, so data-quality edge cases (§14.5) get exercised naturally during development.
10. **Validation instrumentation** (§15) — the logging/labeling hooks should go in now even though the actual calibration pass happens later; retrofitting this after the fact means losing early session data you can't recreate.

If you want, I can next write out the actual pseudocode/code for any specific stage – the filtering/baseline module and the onset state machine are the two most worth getting exactly right before you touch anything else.